

Since **v-if** is a directive, it must be associated with a single element. But what if somebody wants to toggle more than a single element? **v-if** can also help with that easily. This is another interesting fact about the **v-if** directive. It can be applied to the template block even if the user doesn't want to render anything between them. Either a child component or a lot of other elements is present in that block. The final rendered result will not consist of the **<template>** element.

As far as the type of rendering is concerned, **v-if** is considered more accurate because it ensures that everything inside the conditional block, including event listeners and child components, is appropriately destroyed. It also facilitates the recreation of these elements during toggles.

Additionally, if the condition is false on the initial step of rendering, the **v-if** directive will simply not execute. The conditional block will not get rendered until and unless the given condition becomes true for the first time. This feature of **v-if** often makes developers call it 'lazy' as compared to other directives.

#### For example,

- `<!-- This template is conditionally rendering -->`
- `<template v-if="isBool">`
- `<h1>This is a Heading</h1>`
- `<p>This is a paragraph</p>`
- 
- `<!-- A child component -->`
- `<Hello msg="Hello Vue" />`
- `</template>`

### v-else

Like conditional statements of other programming languages, **v-else** can be used along with **v-if** if there is more than one condition. **v-else** conditional rendering is instrumental when the user wants to render between any of the two code blocks. It is essential to keep in mind that **v-else** only gets executed when it appears right after the **v-if** block or a **v-else** if block.

#### For example,

- `<!-- This div is conditionally rendering -->`
- `<div v-if="Math.random() > 0.5">`
- `Now you see me`